Теоретический материал к лабораторной работе №1

Общие принципы работы с командами в Linux:

Большинство команд имеют следующую структуру:

1. На первом месте идёт имя команды (что делать).
2. Далее за именем команды идут ключи (опции), расширяющие и/или модифицирующие поведение команды (как делать).
3. За ключами идут аргументы, на которые направлено действие команды (над чем делать).

Ключи бывают двух типов:

1. Короткие (один латинский символ).
2. Длинные (как правило, длинные ключи являются человеко-читаемыми).

Практически каждая команда имеет опцию --help, ознакомление с результатом которой поможет вам лучше разобраться в этой команде и посмотреть список ее возможных опций.

Некоторые команды могут потребовать от вас использования ***sudo*** (команду superuser do). Это дает этой команде полный доступ к любым действиям на вашем компьютере. Суперпользователь - это специальная учетная запись, часто называемая учетной записью root, с полными правами доступа к системе. Они являются эквивалентом учетной записи администратора Windows. Некоторые обычные учетные записи, называемые “sudoers”, имеют возможность выступать в качестве суперпользователя, используя команду sudo.

Результат выполнения большинства команд отправляется в ***стандартный поток вывода stdout***, который по умолчанию ориентирован на терминал, а ***стандартный поток ввода stdin*** – на клавиатуру. Кроме того, существует и ***стандартный поток ошибок stderr***, который также выводит данные в терминал.

Логично ожидать, что пользователь может управлять потоками:

obj_1 < obj_2 – оператор, который перенаправляет стандартный поток ввода (stdin) операнда слева (obj_1), заменяя ввод по умолчанию (клавиатуру) на чтение данных из объекта справа (obj_2). obj_2 обязательно должен быть пассивным источником данных.

> – перенаправляет стандартный поток вывода на аргумент справа от >.

>> – перенаправляет стандартный поток вывода на объект справа от >> в режиме записи в конец.

2> – перенаправляет стандартный поток ошибок на объект справа от 2> в режиме перезаписи.

2>> – перенаправляет стандартный поток ошибок на объект справа от 2>> в режиме записи в конец.

Базовые команды:

Команда ls позволяет посмотреть содержимое текущей директории.

```
pavel@pavel-VirtualBox:~/Desktop$ ls
```

В приглашающей строке терминала можно увидеть имя текущего пользователя (часть до “@”), имя хоста (часть после “@” и до “:”), а также текущую рабочую директорию. Символ “~” является сокращением пути до домашней директории текущего пользователя /home.

Команда pwd позволяет получить путь до текущей рабочей директории (сокр. от path working directory).

Для перемещения между директориями применяется ***команда cd*** (сокр. от change directory).

```
$ cd директория_назначения
```

Также с командой cd могут быть использованы сокращения

~ – для указания пути к домашней директории текущего пользователя

.. – для указания пути к родительской директории текущего расположения

Для создания новой пустой директории используется ***команда mkdir*** (сокр. от make directory).

```
$ mkdir путь_к_директории
```

Удалить пустую директорию позволит ***команда rmdir*** (сокр. от remove directory).

```
$ rmdir путь_к_директории
```

Создать новый файл можно с помощью ***команды touch***.

```
$ touch путь_к_файлу
```

Удалить файл позволяет ***команда rm*** (сокр. от remove).

```
$ rm путь_к_файлу
```

Посчитать количество строк слов и байт можно с помощью ***команды wc*** (сокр. от word count).

```
$ wc путь_к_файлу
```

Команда echo— это встроенная команда оболочки Bash, которая выводит строку текста в терминал. Она может применяться в различных скриптах, программах и для редактирования конфигурационных файлов.

```
$ echo аргумент
```

Команды head и tail — две наиболее часто используемые команды для работы с текстом в Linux.

Команда head используется для распечатки начальных строк файла. Она считывает файлы с самого начала. Команда tail печатает последние строки файла. Она считывает файлы с конца и выводит конечные строки.

```
$ head -n число_строк файл
```

```
$ tail -n число_строк файл
```

Команда cat выводит содержание файлов в окне терминала. Обратите внимание, что вывод команд можно перенаправлять из терминала в файл или на вход другой команды с помощью операторов управления потоком.

```
$ cat [опции] путь_к_файлу
```

Команда tr переводит, удаляет и сжимает символы из стандартного ввода и записывает результат в стандартный вывод.

```
$ tr [опции] строка_символов
```

Команда sort – это утилита для вывода текстовых строк в определённом порядке. Её можно использовать для сортировки текста из одного или нескольких файлов, или для сортировки вывода какой-либо команды. `$ sort [опции] путь_к_файлу`

Команда uniq предназначена для поиска одинаковых строк в массивах текста. При этом с найденными совпадениями пользователь может совершать множество действий – например, удалять их из вывода либо наоборот, выводить только их. *Она сравнивает только строки, которые находятся рядом.* То есть, если две строки, состоящие из одинакового набора символов, идут подряд, то они будут обнаружены, а если между ними расположена строка с отличающимся набором символов – то не будут поэтому перед сравнением желательно отсортировать строки с помощью sort.

```
$ uniq [опции] путь_к_файлу
```

N.B. Мы рассмотрели лишь базовые возможности приведенных выше команд. Узнать об их параметрах и о том, как вы можете модифицировать их поведение можно с помощью флага ***--help***. Для получения руководства можно воспользоваться ***командой man.*** Она принимает в качестве аргумента название команды и отображает страницу руководства.

```
$ man wc
```

Обработка данных по каналу

Канал командной строки | (вертикальная черта), является одним из самых полезных операторов, которые только можно себе представить. Он передает выходные данные команды с левой стороны канала в команду с правой стороны канала через специальное соединение.

```
$ env | wc -l
```

Возможность объединения нескольких команд Linux в терминале и использования их в качестве конвейера, когда каждая следующая команда получает вывод предыдущей - очень мощный и гибкий инструмент. Но команды можно объединять не только так. С помощью ***утилиты xargs*** вывод предыдущей команды можно передать в аргументы следующей.

```
$ первая_команда | xargs [опции] вторая_команда аргументы
```

Пример команды создающей файлы с именами из файла filenames.txt:

```
$ cat filenames.txt | xargs touch
```

Регулярные выражения

Регулярное выражение – это последовательность символов указывающая поисковый шаблон.

Что нужно знать:

- Регулярные выражения чувствительны к регистру
- При необходимости использовать символ, который является специальным вам нужно его экранировать. Пример: чтобы ваш шаблон искал “?” вам необходимо добавить в регулярное выражение \?
- Написанный вами шаблон применяется ко всей строке, если не указаны якоря

Основной набор спецсимволов, можно разбить на группы:

Символьные группы:

. – обозначает 1 любой символ кроме символа начала новой строки

[abc] – обозначает 1 символ из заданной внутри скобок группы

[^abc] – обозначает 1 символ, который может быть любым кроме a, b или c.

[a-z] – обозначает 1 символ из заданного внутри скобок отрезка по кодам ASCII

(a | b) – обозначает выбор между a или b. Может применяться с символьными группами.

Указатели количества

* – 0 или более символов

+ – 1 или более символов

{X} – ровно X символов.

{X,Y} – от X до Y символов

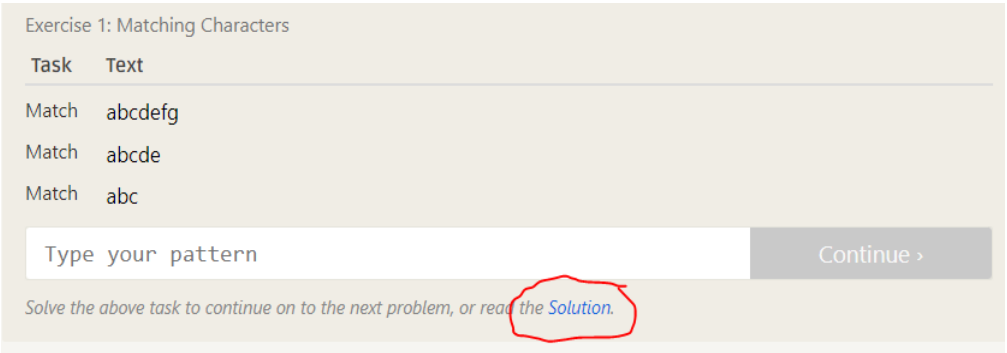
{X,} – X и более символов

Якоря

^ – устанавливает якорь на начало строки. Шаблон будет проверяться только для начала строки.

$ – устанавливает якорь на конец строки. Шаблон будет проверять оканчивается ли после него строка.

С хорошим руководством по работе с регулярными представленным в виде группы уроков с примерами можно ознакомится здесь: https://regexone.com/lesson/introduction_abcs. Смело можете попробовать написать регулярные выражения до 10-го урока включительно. Обратите внимание, если у вас не получается написать регулярное выражение, то можно посмотреть ответ кликнув по решению как показано на скриншоте ниже.



Для отладки написанных вами регулярных выражений можно использовать отладчик представленный на https://regex101.com/. Некоторые спецсимволы для регулярных приложений могут отсутствовать в различных утилитах или иметь иной символьный вид. Тут вам на помощь может прийти: https://cheatography.com/davechild/cheat-sheets/regular-expressions/

grep и sed

Команда grep —находит на вводе строки, отвечающие заданному регулярному выражению, и выводит их, если вывод не отменён специальным ключом.

```
$ grep [опции] шаблон путь_к_файлу
```

При работе в режиме потока:

```
$ команда | grep [опции] шаблон
```

Опции - это дополнительные параметры, с помощью которых указываются различные настройки поиска и вывода, например, количество строк или режим инверсии. Шаблон - это любая строка или регулярное выражение, по которому будет выполняться поиск. Имя файла или папки - это то место, где будет выполняться поиск. Как вы увидите дальше, grep позволяет искать в нескольких файлах и даже в каталоге, используя рекурсивный режим.

В параметрах вы найдете флаги для режима базовых регулярных выражений (флаг -G) и режим расширенных регулярных выражений (флаг -E). Обратите внимание, что последний позволяет работать с регулярными выражениями, которые содержат спецсимволы:

+ — одно или больше повторений предыдущего символа;

? — ноль или одно повторение предыдущего символа;

{n,m} — повторение предыдущего символа от n до m раз;

| — позволяет объединять несколько паттернов.

Команда sed – это потоковый редактор текста, работающий по принципу замены. Её можно использовать для поиска, вставки, замены и удаления фрагментов в файле. Позволяет редактировать файлы, не открывая их.

`$ sed [опции] -e команды файл`

Подробнее о возможностях sed вы можете узнать с помощью флага --help.

Bash Scripting

Shell script - это текстовый файл, содержащий последовательность команд для операционной системы на базе Unix. Он называется сценарием, потому что объединяет последовательность команд, которые в противном случае пришлось бы вводить на клавиатуре по одной, в один набор. Большинство оболочек имеют свой собственный язык сценариев, в каждом из которых есть переменные, поток управления и свой собственный синтаксис. ***Bash scripting*** - это тип сценариев оболочки.

Что отличает shell-скрипты от других скриптовых языков, так это то, что они оптимизированы для выполнения задачи, связанные с командной оболочкой. Создание конвейеров команд, сохранение результатов в файлах и чтение со стандартного ввода- это примитивы сценариев командной оболочки, которые упрощают их использование по сравнению с другими языками сценариев. (Например, если вы хотите запустить команду cd в Python, вам нужно будет импортировать библиотеку операционной системы, а затем вызвать chdir из этой библиотеки.)

Содержание вашего первого bash script с именем hello.sh может быть таким:

```
1 #!/ usr/ bin / env bash
2 echo " Hello world !"
```

Первая строка в этом скрипте (`#!/usr/env/ bash`) называется shebang, или резкое восклицание.

Существует несколько различных способов написания вашего shebang, таких как `#!/usr/bin/env bash` и `#!/bin/bash` . Рекомендуестя всегда использовать первый, поскольку это повышает совместимость вашего скрипта.

Для запуска скрипта потребуется превратить его в исполняемую программу, а затем запустить ее. Сначала вам нужно преобразовать файл со скриптом в исполняемый файл с помощью команды

chmod (change mode). Эта команда используется для изменения прав доступа к файлу, например, для того, чтобы сделать файл исполняемым. Вы можете сделать это следующим образом:

```
$ chmod +x hello.sh
```

Чтобы запустить скрипт, вам нужно просто запустить программу с помощью ./hello.sh

Как и в других языках программирования, в скриптах bash также есть директивы потока управления, такие как if, for, while и case.

Синтаксис для написания инструкции if в bash выглядит следующим образом:

```
1 #!/ usr/ bin / env bash
2
3 if [ Условие ]
4 then
5 # do something
6 fi
```

Пример скрипта выводящего информацию, если число больше чем 100:

```
1 #!/ usr/ bin / env bash
2
3 num=101
4 if [ $num - gt 100 ]
5 then
6 echo " That ’s a big number !"
7 fi
```

Обратите внимание на наличие пробелов между скобками и выражением отвечающем за условие if. Также после завершения тела if ставится команда fi.

Синтаксис для добавления цикла while в сценарий bash следующий:

```
1 #!/ usr/ bin / env bash
2
3 while [ CONDITION ]
4 do
5 # выполнение команд в теле цикла
6 done
```

Пример сценария выводящего в консоль числа от 0 до 99:

```
1 #!/ usr/ bin / env bash
2
3 num=0
4 while [ $num - lt 100 ]
5 do
6 echo $num
7 num =$(( num +1) )
8 done
```

Чтобы увеличить значение num , мы используем двойные круглые скобки ((..)) для арифметического вычисления. Внутри двойных круглых скобок мы можем увеличить значение num на 1.

Чтобы объявить цикл for в bash, мы можем объявить либо цикл for на основе индекса, либо цикл for на основе диапазона. Чтобы объявить цикл for на основе индекса, мы будем использовать следующий синтаксис:

```
1 #!/ usr/ bin / env bash
2
3 for имя_переменной in 1 2 3 .. N
4 do
5 # do something
6 done
```

Пример сценария, выполняющего ровно тот же функционал, что и скрипт с циклом while:

```
1 #!/ usr/ bin / env bash
2
3 num =0
4 for i in {0..99}
5 do
6 echo $num
7 num =$(( num +1) )
```

```
8 done
```

Функции

В bash мы будем ссылаться на аргументы, передаваемые в функцию, в зависимости от их положения. Например, мы могли бы использовать значение самого первого аргумента и ссылаться на него с помощью $1. В bash переменные $1 - $9 относятся к аргументам скрипта. Переменная $0 относится к имени скрипта.

Пример функции, которая создает новую директорию и сразу переходит в нее:

```
1 #!/ usr/ bin / env bash

2

3 make_and_enter () {

4 mkdir -p "$1"

5 cd "$1"

6 }

7 # чтобы вызвать функцию напишите ее имя и передайте аргументы, если они требуются

8 make_and_enter new_folder
```

Мы также можем определить строки в скрипте bash. Если мы хотим определить строковый литерал, мы будем использовать одинарные кавычки: '$x'. Если мы хотим определить строку, которая допускает замену, мы будем использовать двойные кавычки: "$x".

Двойные кавычки позволят заменить переменные:

```
1 x=foo

2 echo ’$x ’

3 # prints $x

4 echo "$x"

5 # prints foo
```

Символ внутри скрипта # позволяет оставлять комментарии, так как он указывает, что содержащая его строка не должна исполняться.

Подстановка команд - еще одна полезная функция сценариев bash. Возможно, вы захотите запустить команду, а затем использовать ее выходные данные в качестве переменной для какого-либо другого фрагмента кода. Вот пример сценария, который использует подстановку команд:

```
1 #!/ usr/ bin / env bash

2

3 for element in $( ls ~/ Desktop )
```

```
4 do
5 echo " Desktop contains file named $element "
6 done
```

Сравнения

В Bash проводится различие между сравнениями чисел и сравнениями строк. Чтобы сравнить числа в скрипте bash, используйте следующее:

- a -eq b для проверки, равны ли a и b
- a -ne b для проверки, не равны ли a и b
- a -gt b для проверки, не больше ли a чем b
- a -ge b для проверки, что a больше или равно b
- a -lt b для проверки, что a меньше или равно b

Чтобы сравнить строки в bash-скрипте, используйте:

- s1 = s2 для проверки, равно ли значение s1 значению s2
- s1 != s2 для проверки, не равно ли значение s1 значению s2
- s1 < s2 для проверки, не меньше ли значение s1 значения s2 в лексикографическом порядке
- s1 > s1 для проверки, не больше ли значение s1 значения s2 в лексикографическом порядке
- -n s1 для проверки, имеет ли s1 длину больше 0
- -z s1 для проверки, имеет ли s1 длину 0